



计算机图形学与虚拟现实

冼楚华

Email: chhxian@scut.edu.cn

华南理工大学计算机科学与工程学院



课程信息

- 授课老师姓名：冼楚华
- Email: chhxian@scut.edu.cn 
- 个人主页: <https://chuhuaxian.github.io/>
- QQ: 89071086 （比较少用，非急事请不要私聊）
- 办公室: B3-202-2
- 课程QQ群（见二维码）



群名称: 2025计算机图形学@SCUT
群 号: 134660465

主要内容

- Introduction
- 光照模型(illumination models)
- 明暗处理(Polygon shading)
- 光线跟踪(Raytracing algorithm, Section 12.3)
- 纹理映射

- <https://haokan.baidu.com/v?vid=13080958122684049492&pd=bjh&fr=bjhauthor&type=video>
- <https://www.bilibili.com/video/BV1FJ41117wK/>
- <https://www.bilibili.com/video/BV1fh411r7c7/>

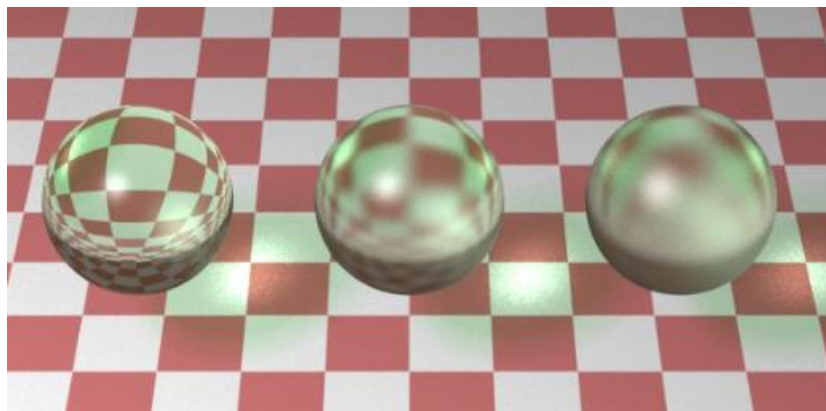
1 Whitted模型 (global illumination model)

- 局部光照明模型的局限

- 只考虑从光源直接发出的光线对表面光亮度的贡献
- 不考虑光线在物体之间的相互反射和透射

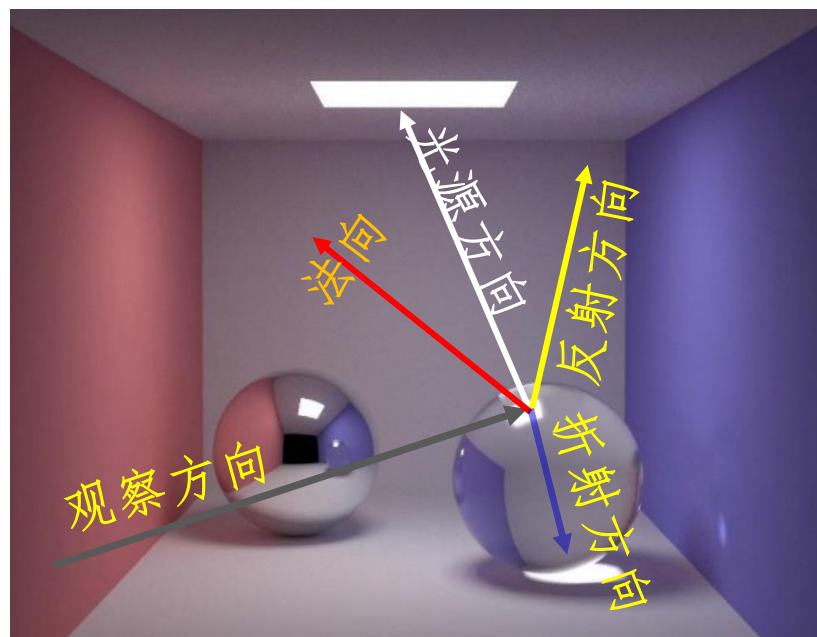
- Whitted模型：全局光照明模型

- 某个点的光强与整个场景相关
- 表面之间的镜面反射
- 透明物体的透射



Whitted模型光照构成

- 物体表面点P的光亮度来自三个方面
 - 由光源直接照射引起的漫反射和镜面反射光亮度 (local illumination)
 - 场景间接引起的镜面反射光 (specular light)
 - 透射光 (transmission light)



Whitted全局光照(illumination model)

$$I = I_c + K_s I_r + K_t I_t$$

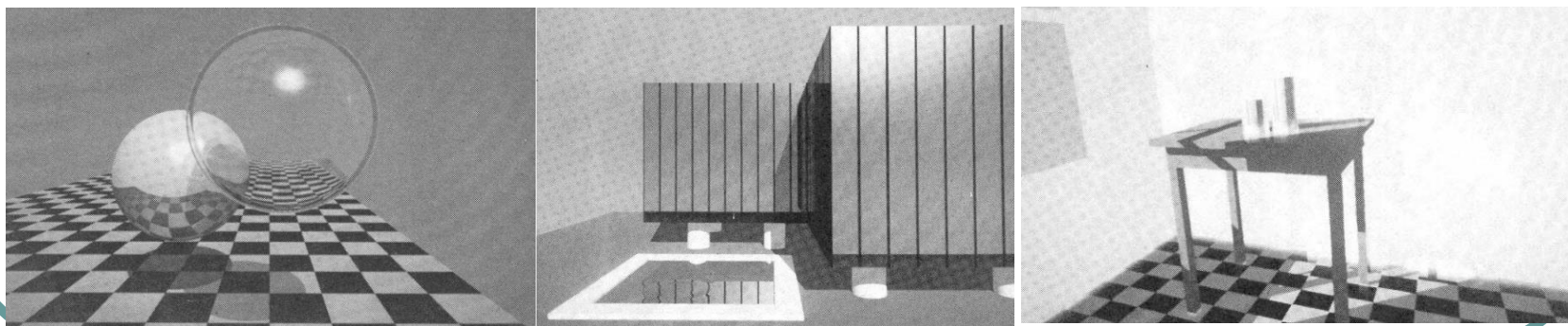
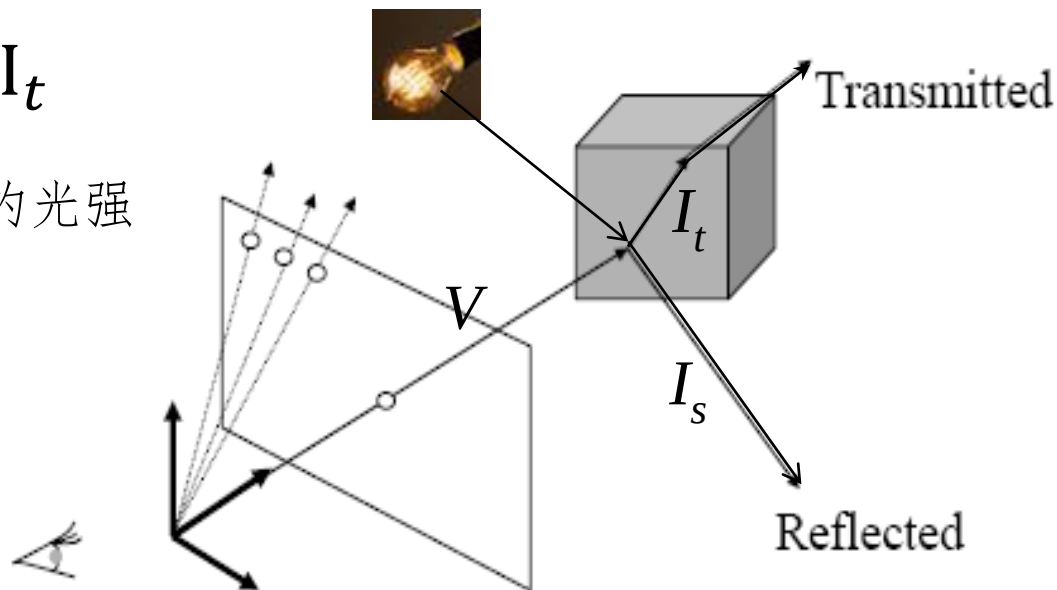
I_c : 局部光照模型计算的光强

I_r : 环境的镜面反射光

I_t : 环境的规则透射光

k_s : 表面的镜面反射率

k_t : 表面的透射率

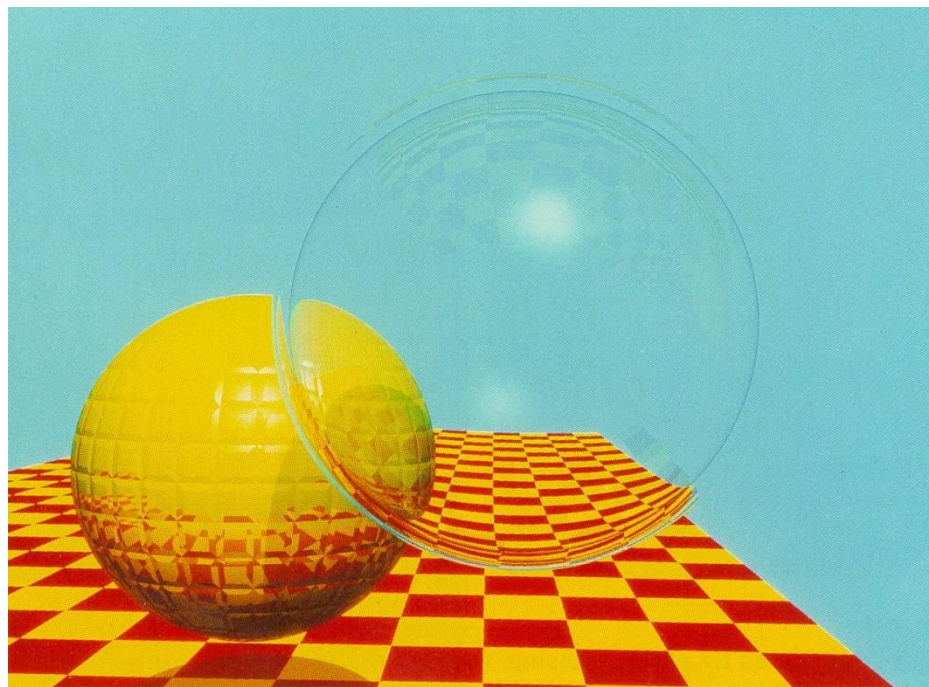


Whitted T. An improved illumination model for shaded display[C]//Proceedings of the 6th annual conference on Computer graphics and interactive techniques. 1979: 14.

Whitted模型—求解算法

- 光线跟踪

Whitted T, **An improved illumination model for shaded display**, Communications of the ACM, 1980, 23(6):343-349.



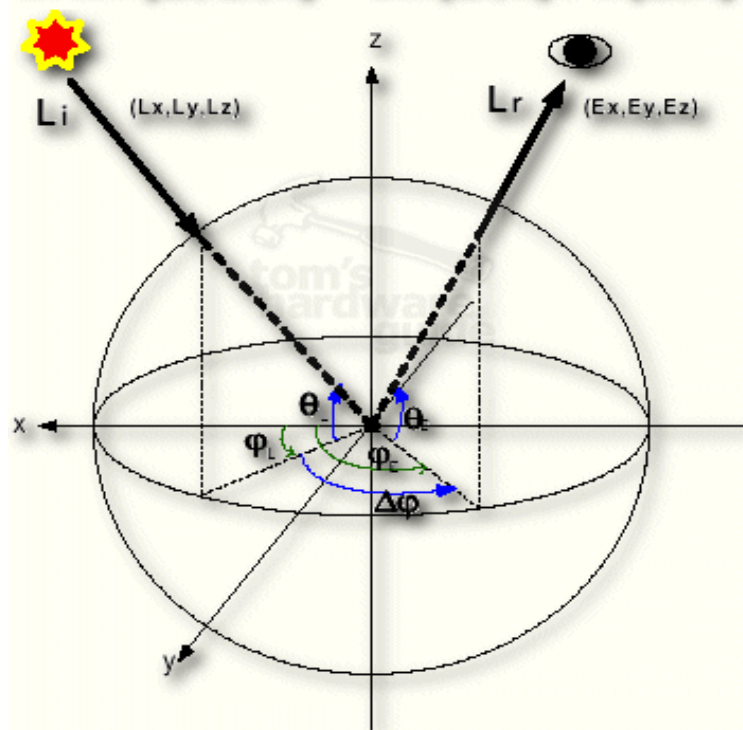
6 光照明模型的进一步完善

- 物体表面反射光线亮度可由一个各向异性函数描述(如BRDF, **Bidirectional reflectance distribution function**)

$$\text{BRDF}(\omega_i, \omega_r, \lambda) = dL_r(\omega_r, \lambda) / L_i(\omega_i, \lambda)$$

- 相关参数

- 入射光线方向
- 物体表面法向
- 观察方向
- 入射位置
- 出射位置...

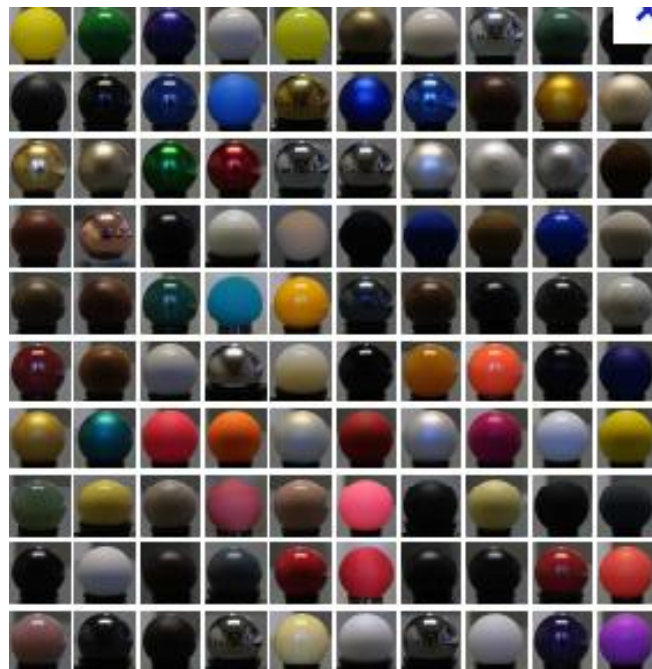




光照明模型的进一步完善

● 光照明模型分类

- 基于经验的简单光照明模型
 - Phong模型
 - Blinn-Phong模型(简化Phong)
- 基于物理的光照明模型
 - Cook-Torrence模型
 - 辐射度光照模型
- 渲染方程Rendering equation

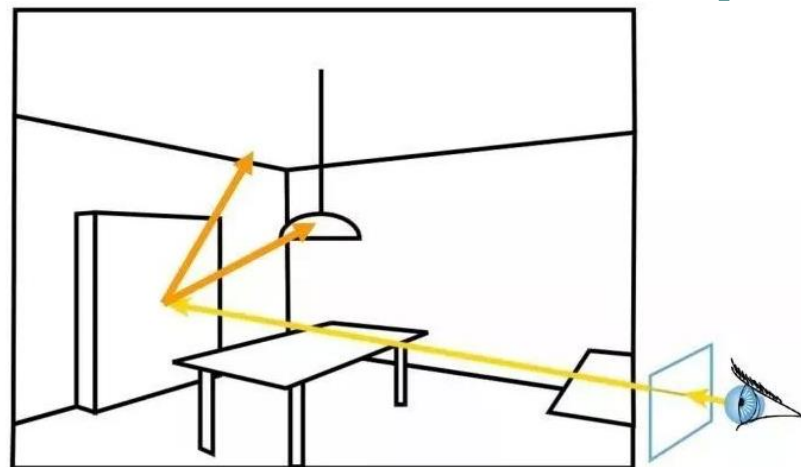


1 光线投射(Ray casting algorithm, RC)

- 一种光线/视线(Ray)与曲面相交测试方法
- 光线投射算法流程

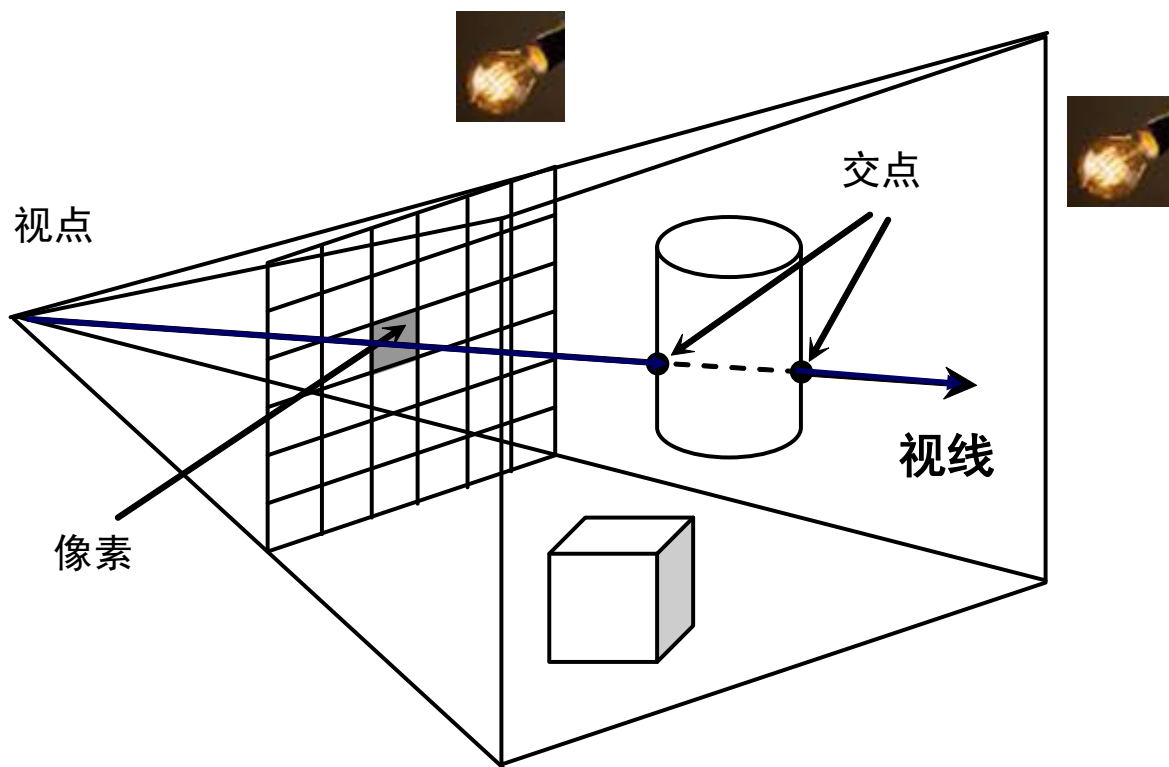
对屏幕中的每个像素：

- Step1: 视点与象素连线与所有物体求交
- Step2: 取离视点最近的交点
- Step3: 交点与所有光源连线作遮挡测试 $m_i = 1$ 不遮挡, 0 遮挡
- Step4: 局部光照模型计算交点处颜色



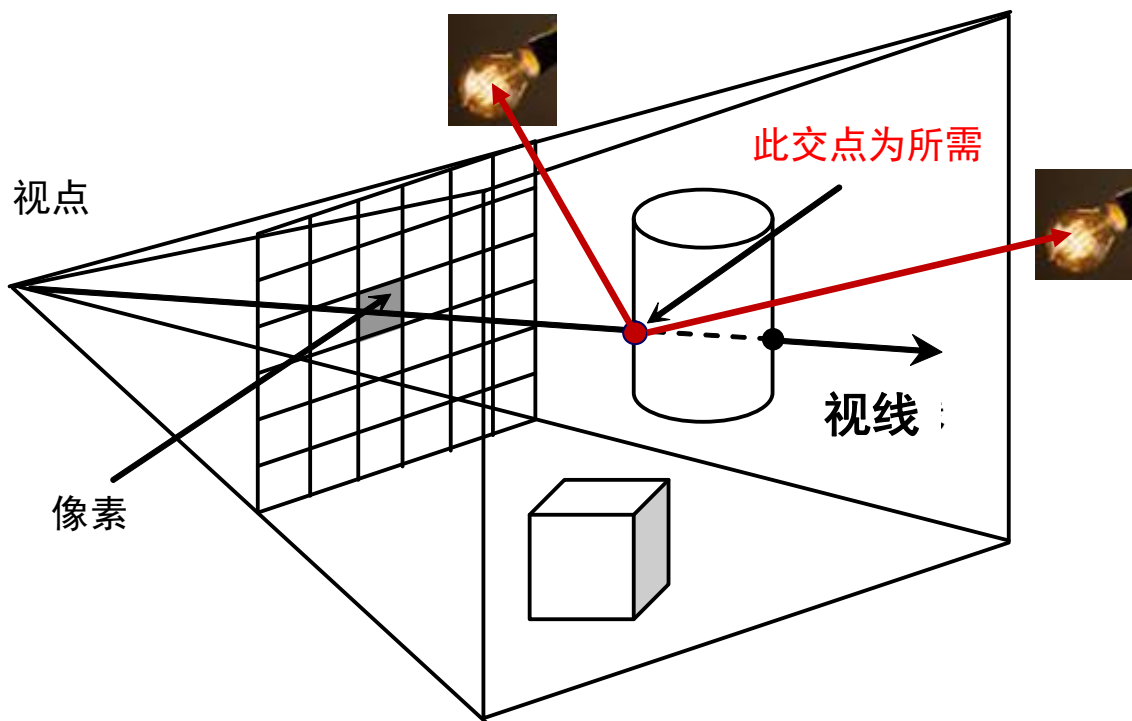
光线投射—步骤

- Step1: 求视线与场景的所有交点(Find intersections)



光线投射--步骤

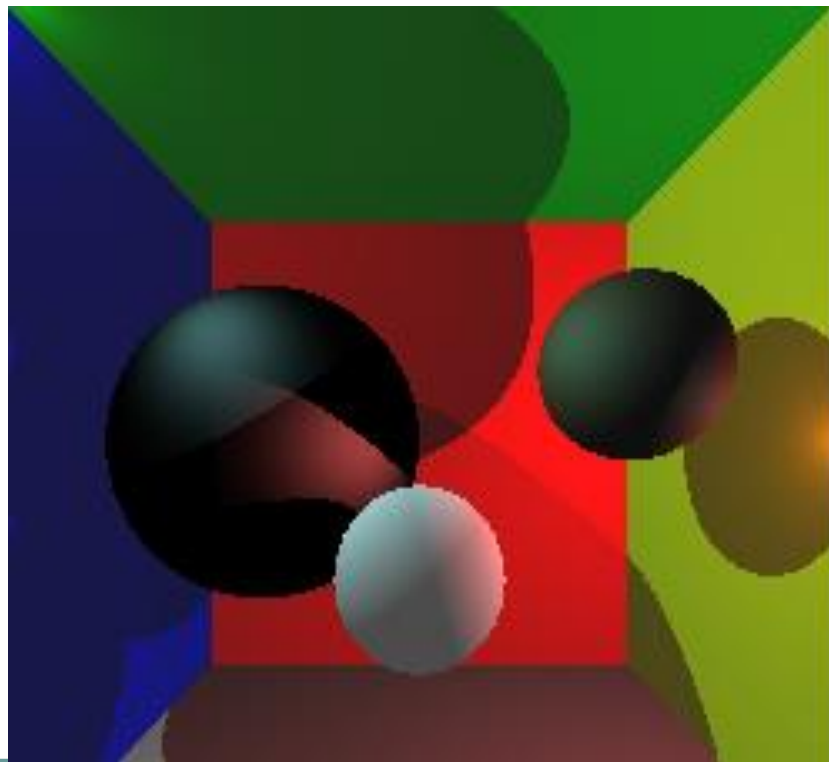
- Step2: 找到最近交点(消隐)
- Step3: 与光源连线, 遮挡测试



光线投射--步骤(4)

- Step 4: 计算局部光照

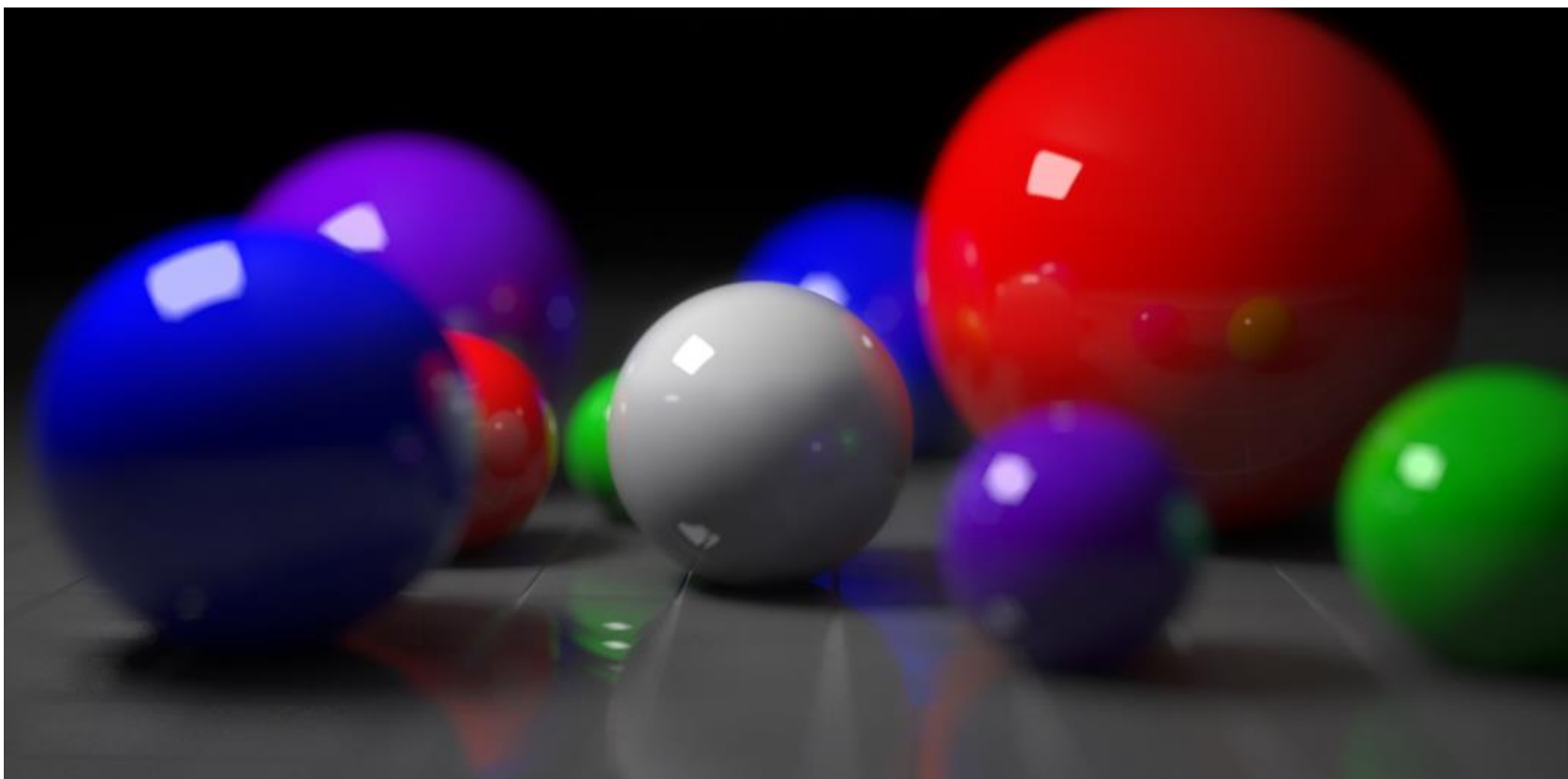
$$I = K_a I_a + \sum_{i=1}^m I_i (K_d \cos \alpha_i + K_s \cos^n \gamma_i)$$



问题

- 光线投射是图像空间还是对象空间方法？
- 场景中有参数曲面时，是否必须三角剖分？
- 光线投射如何消隐？
- 光线投射如何产生阴影？

2 光线跟踪(Ray Tracing)



光线跟踪算法(Ray tracing algorithm)

- 一个重要的全局光照算法(An important photorealistic algorithm)
 - 受光线投射算法启发(ray casting algorithm)
- 特点
 - 思想简单(Simple)
 - 计算全局光照, 更强的真实感效果(More realistic)
 - 适合镜面反射、折射场景
 - 计算代价高(High cost)

● 光照模型: 全局光照

$$I = I_c + K_s I_r + K_t I_t$$

● 逐像素计算

- 通过追踪光线收集进入视点的光强值

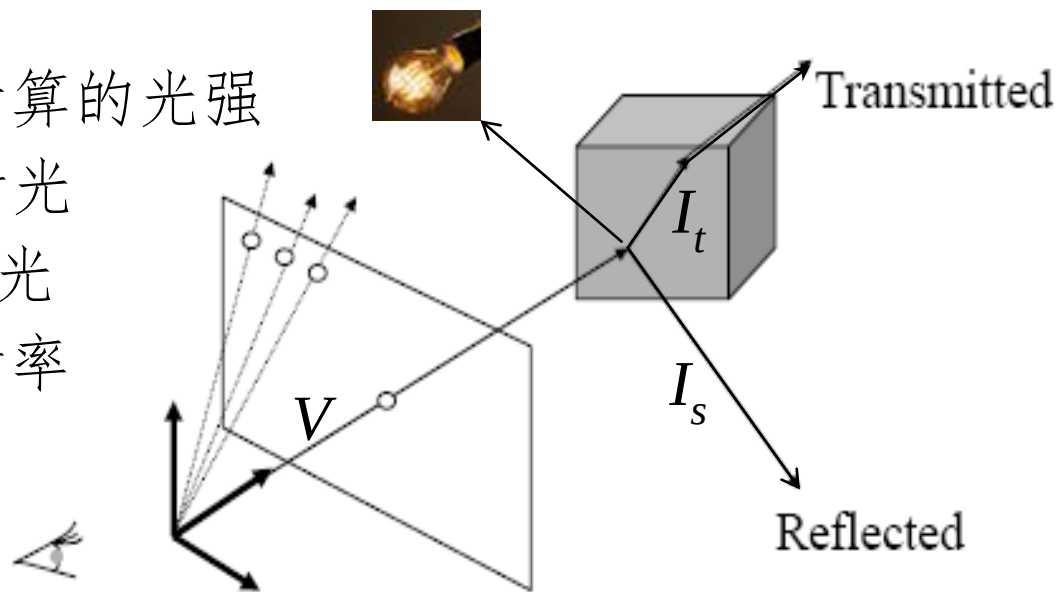
I_c : 局部光照模型计算的光强

I_r : 环境的镜面反射光

I_t : 环境的规则透射光

k_s : 表面的镜面反射率

k_t : 表面的透射率



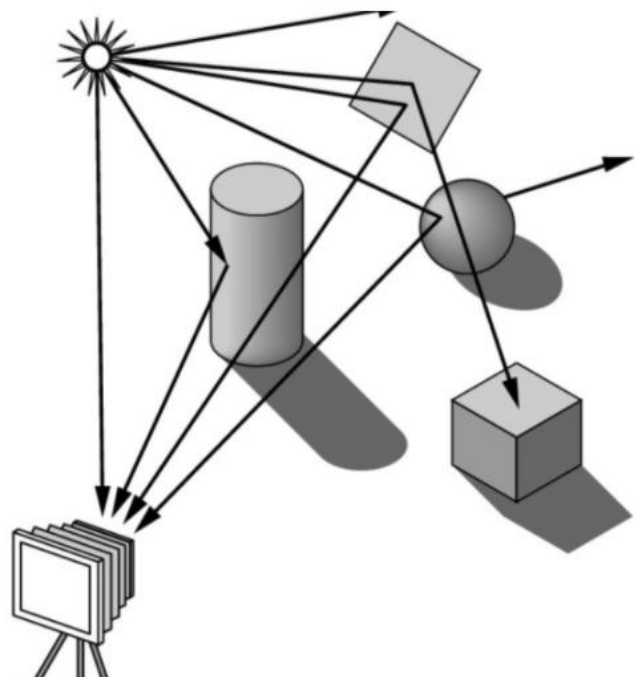
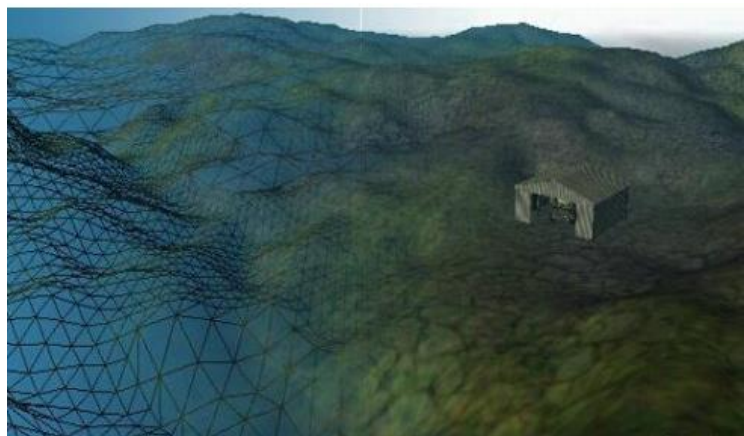
光线跟踪(Ray Tracing)---逆向思想

- 逆向追踪

- 追踪视线方向来的光线，而非光源发出的每条光线

- 三种光线

- 来自光源；
 - 反射方向(Reflection)；
 - 折射(Refraction)



3 光线跟踪算法流程

对指定分辨率图像的每个像素 {

Step 1 引过视点的射线R

Step 2 计算R跟所有物体的交点

Step 3 选取离视点最近交点P

Step 4 计算P点局部光照 I_c

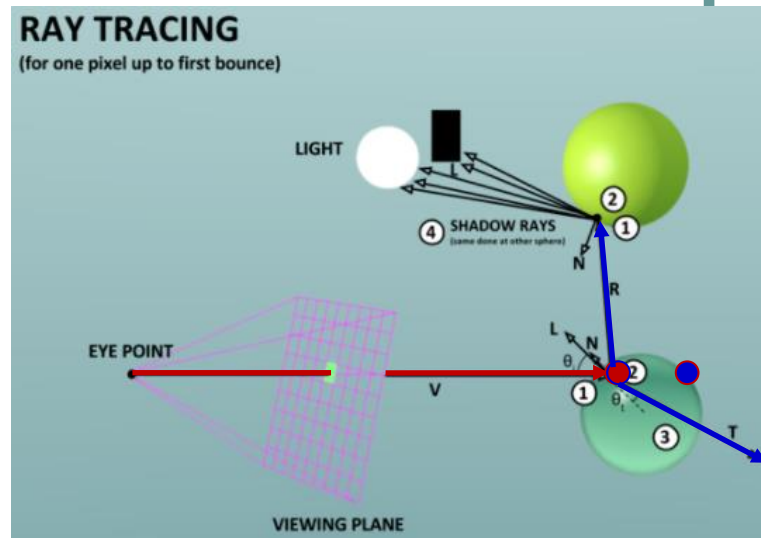
Step 5 从P点发出反射线 R_r

Step 6 从P点发出折射线 R_t (透明)

Step 7 递归地计算 I_r 和 I_t

Step 8 求和 $I(p) = I_c + k_s I_r + k_t I_t$

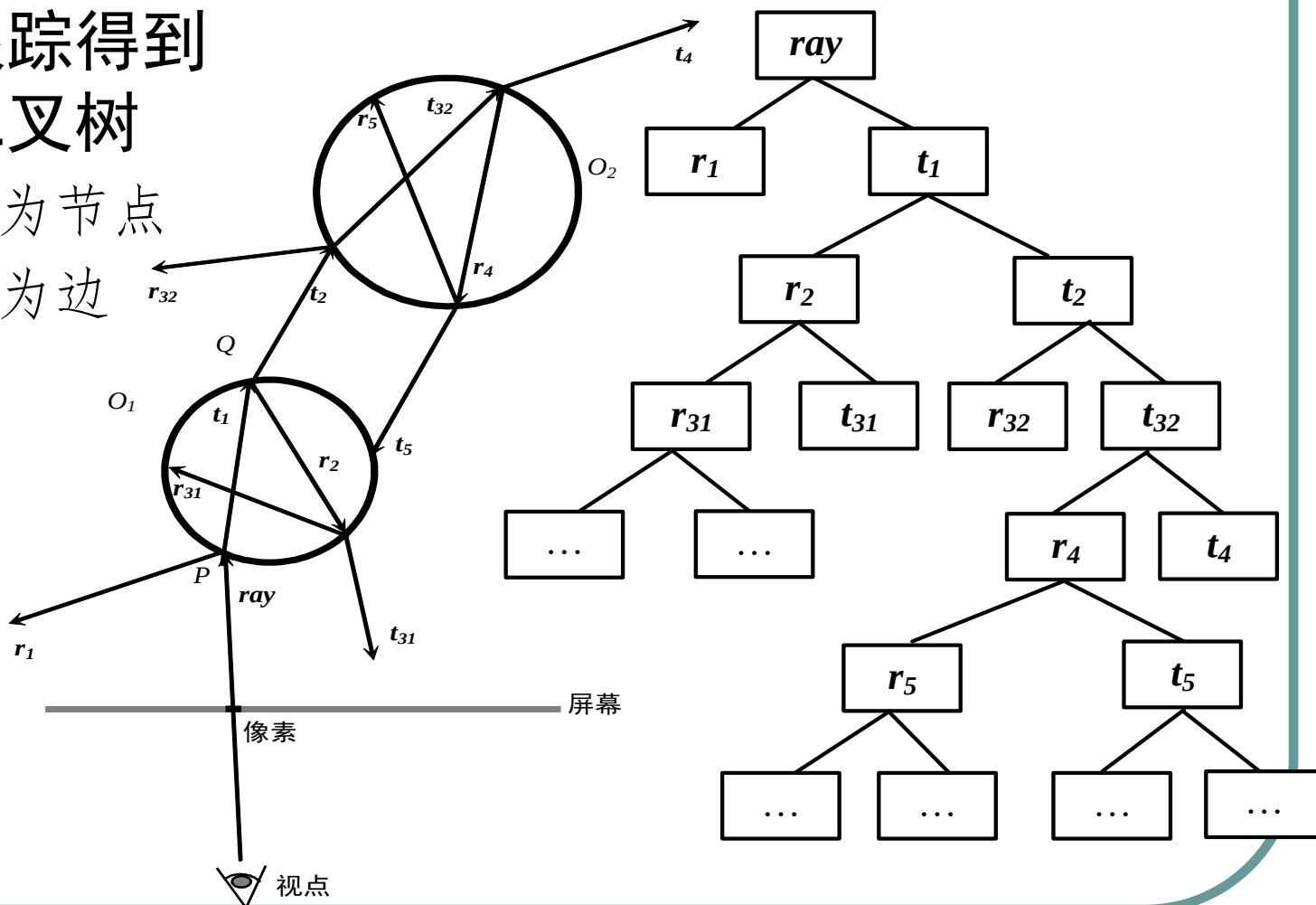
}



光线树(Ray tree of a ray)

- 光线跟踪得到一棵二叉树

- 交点为节点
- 光线为边



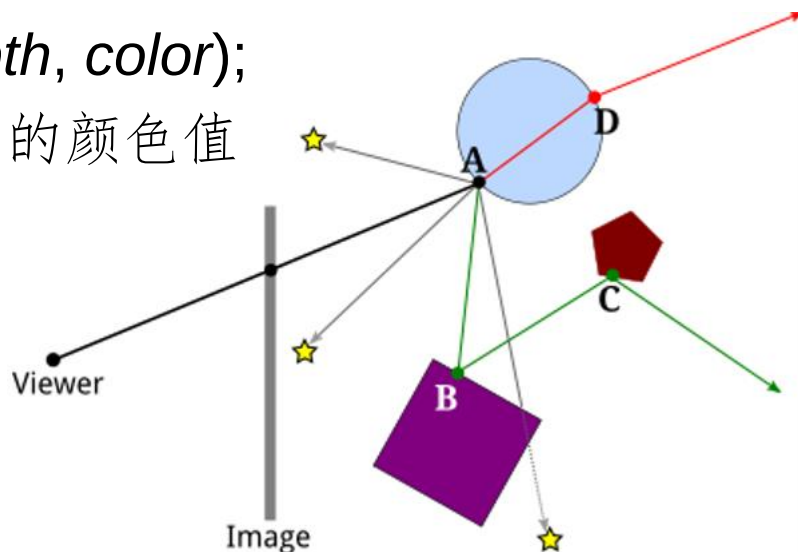
光线跟踪递归终止条件

- 满足如下三个条件之一，不再发射光线
 - 射线不与任何物体或只与漫反射面相交
 - 射线的贡献太小
 - 射线达到最大递归深度



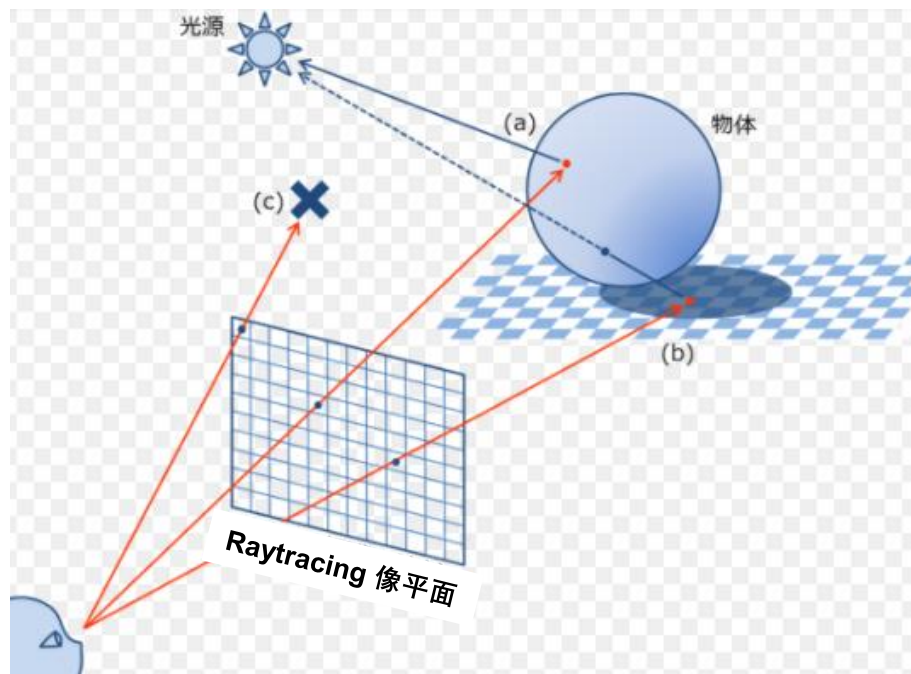
4 伪代码 (1)----遍历所有像素

```
Whitted_Illumination_Model( ) {  
    for(each pixel) {  
        create ray R from viewpoint V to the pixel;  
        depth = 0; //递归深度  
        ratio = 1.0; //当前光线的衰减系数, 1.0表示无衰减  
        RayTracer(R, ratio, depth, color);  
        //color是光线跟踪返回的颜色值  
        pixel  $\leftarrow$  color;  
    }  
}
```



伪代码 (2)----光线跟踪器

```
RayTracer(R, ratio, depth, color) //说明: 光线跟踪子函数
{
    // color用于传回结果
    if(ratio < THRESHOLD) {
        //终止条件2: 衰减小于阈值
        color  $\leftarrow$  0; return;
    }
    if(depth > MAXDEPTH) {
        //终止条件3: 递归到最大次数
        color  $\leftarrow$  0; return;
    }
    //continued
```



伪代码 (3)----计算交点和局部光照

$P \leftarrow \text{FindNearestIntersection}(R)$; //计算 R 的可见点 P

if($P == \text{NULL}$) { // 终止条件1

$color \leftarrow 0$; //置为黑色

 return;

}

$local_color \leftarrow \text{PhongModel}(P)$;

// Ray casting(从 P 发射射线

// 到每个点光源测试是否遮挡



伪代码 (4)----计算反射和透射光线

```
if(交点 $P$ 所在的表面为光滑镜面) {  
    计算反射光线 $R_r$ ;  
    //递归调用!  
    RayTracer( $R_r$ ,  $k_s*ratio$ ,  
         $depth+1$ ,  $reflected\_color$ );  
}  
if(交点 $P$ 所在的表面为透明表面) {  
    计算透射光线 $R_t$ ;  
    //递归调用!  
    RayTracer( $R_t$ ,  $k_t*ratio$ ,  
         $depth+1$ ,  $transmitted\_color$ );  
} // continued
```



伪代码 (5)----计算全局光照

//Whitted光照模型

$color = \mathbf{local_color} + k_s * \mathbf{reflected_color} + k_t * \mathbf{transmitted_color};$

} //Raytracer结束



在OPENGL环境下如何实现？请参考：Ray Tracing in One Weekend by Peter Shirley

光线跟踪细节详释

- 如何表示射线
- 射线与几何体的求交
- 反射方向估计
- 折射方向估计
- 阴影生成

细节1：射线表示(Ray)

- 射线的参数表示(Parametric representation)

$$\mathbf{R}(t) = \mathbf{P} + t \mathbf{D}$$

即：

$$x(t) = x_P + tx_D,$$

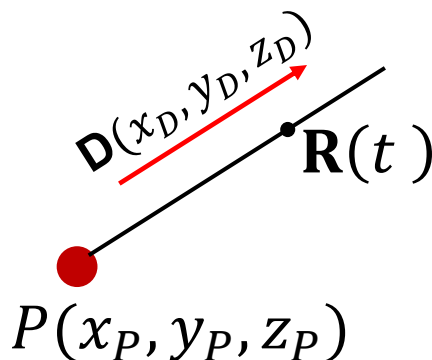
$$y(t) = y_P + ty_D,$$

$$z(t) = z_P + tz_D.$$

- $\mathbf{P} = (x_P, y_P, z_P)$: 起点

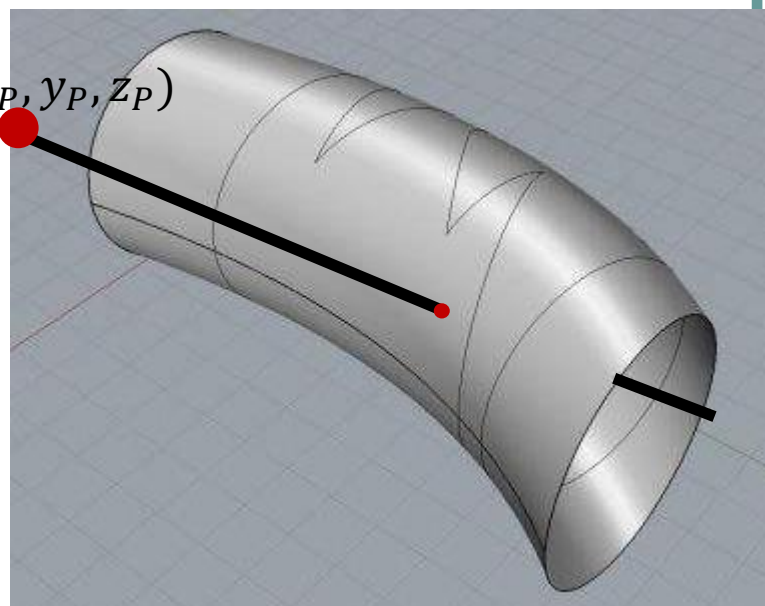
- $\mathbf{D} = (x_D, y_D, z_D)$: 射线方向(单位矢量)

- $t \begin{cases} > 0 & \text{点在光线的正方向} \\ = 0 & \text{起点P} \\ < 0 & \text{光线负方向, 交点无效} \end{cases}$



细节2：线面求交的一般形式

- 光线方程： $\mathbf{R}(t) = \mathbf{P} + t \mathbf{D}$
- 物体表面的表示
 - 隐式方程 $f(x, y, z) = 0$;
 - 参数方程 $(x, y, z) = \mathbf{g}(u, v)$;
- 线面求交
 - 隐式表示需解单未知量方程
 - $f(\mathbf{P} + t \mathbf{D}) = 0$
 - 参数表示需解三元方程
 - $\mathbf{R}(t) = \mathbf{g}(u, v)$
 - 弊端：复杂度高、误差累积、不收敛



细节2.1: 光线与球面求交(Ray and sphere)

- 球面隐式方程

$$(x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 - r^2 = 0$$

- 射线方程 $x(t) = x_P + tx_D, y(t) = y_P + ty_D, z(t) = z_P + tz_D$.

- 代入球面方程得到 t 的二次方程

$$At^2 + Bt + C = 0$$

其中 $A = x_D^2 + y_D^2 + z_D^2 = 1$ (假设光线方向为单位向量)

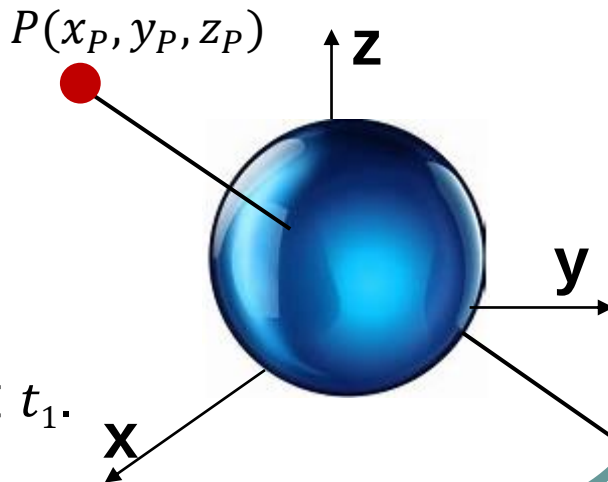
- 解一元二次方程, 得到交点参数值 $P(x_P, y_P, z_P)$

$$t_1 = \frac{1}{2}(-B + \sqrt{B^2 - 4C});$$

$$t_2 = \frac{1}{2}(-B - \sqrt{B^2 - 4C}).$$

- 三种情况:

① $t_2 < t_1 < 0$; ② $t_2 < 0 \leq t_1$; ③ $0 \leq t_2 < t_1$.



细节2.2：光线与三角形求交(Ray and triangle)

- 问题描述：求三角形与射线的交点

- 三角形： $v_i = (x_i, y_i, z_i), i = 1, 2, 3$.
- 射线： $P = (x_p, y_p, z_p), D = (x_r, y_r, z_r)$

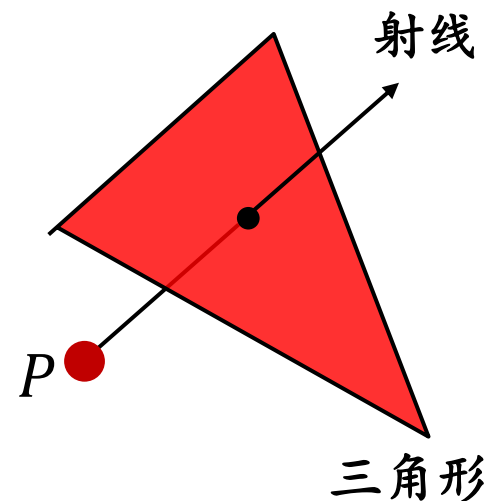
- 基本方法

- 求射线与三角形所在平面的交点
- 判断交点是否位于三角形内部

- 平面方程求解

法向
$$\mathbf{n} = \frac{(\mathbf{v}_2 - \mathbf{v}_1) \times (\mathbf{v}_3 - \mathbf{v}_1)}{|(\mathbf{v}_2 - \mathbf{v}_1) \times (\mathbf{v}_3 - \mathbf{v}_1)|}$$

方程
$$[(x, y, z) - \mathbf{v}_1] \cdot \mathbf{n} = 0$$



细节2.2.1 光线与三角形所在平面求交

- 一般地，平面方程可写为

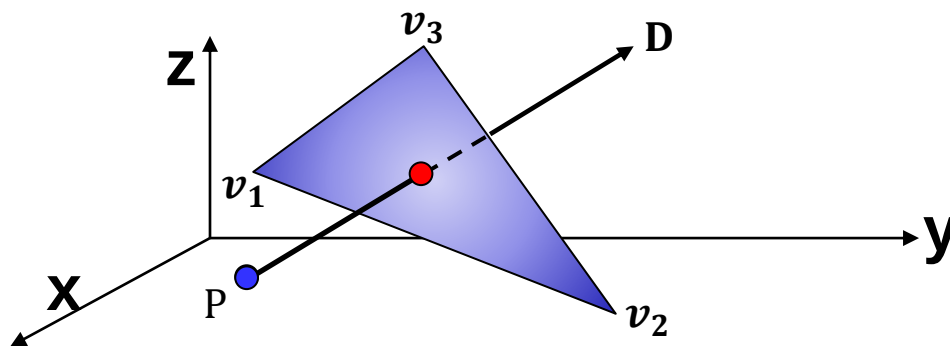
$$ax + by + cz + d = 0, \mathbf{n} = (a, b, c)$$

- 光线方程 $\mathbf{R}(t) = P + t\mathbf{D}$ 代入平面方程得

$$(P + t\mathbf{D}) \cdot \mathbf{n} + d = 0$$

$$t^* = -(P \cdot \mathbf{n} + d) / (\mathbf{D} \cdot \mathbf{n})$$

- $\mathbf{D} \cdot \mathbf{n} = 0$: 光线与平面平行, 无交点
- $t^* < 0$: 交点位于射线之外, 无效, 舍弃
- 代入光线方程得到光线与平面的交点 $I = \mathbf{R}(t)$



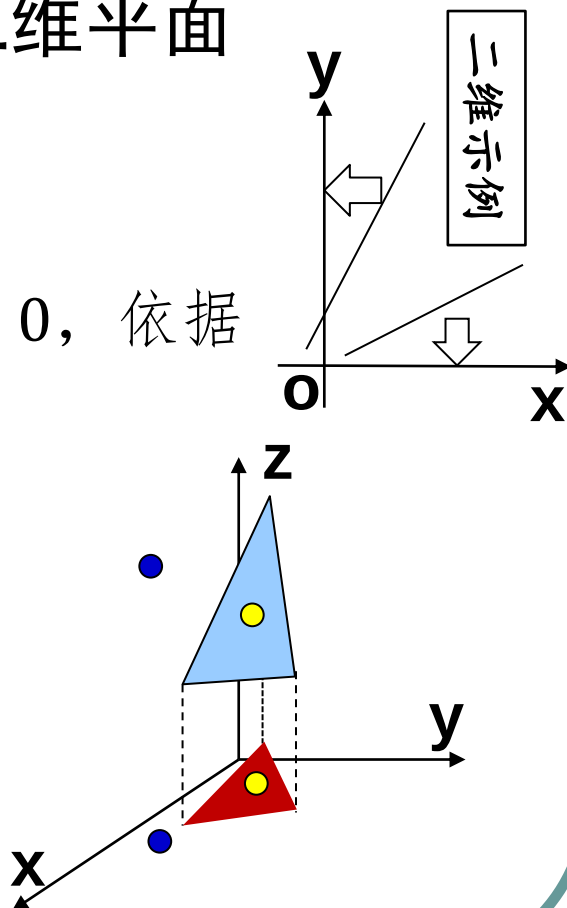
细节2.2.2 判断交点是否位于三角形内

- 将3D三角形内的点判别转化到二维平面

- 理想的投影平面： oxy, oyz, ozx .

- 投影方法

- 设平面方程为 $Ax + By + Cz + D = 0$ ，依据 $|A|, |B|, |C|$ 大小决定投影平面
- 若 $|A| = \max(|A|, |B|, |C|)$ ，取 yoz
- 若 $|B| = \max(|A|, |B|, |C|)$ ，取 zox
- 若 $|C| = \max(|A|, |B|, |C|)$ ，取 xoy



例：光线与三角形相交判断

- 三角形 $(3,1,3), (10,1,3), (3,8,3)$

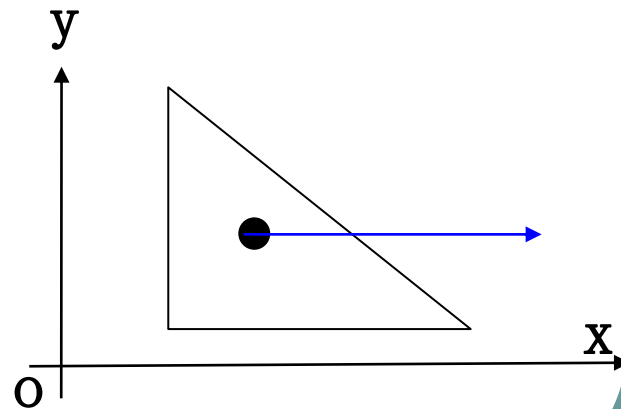
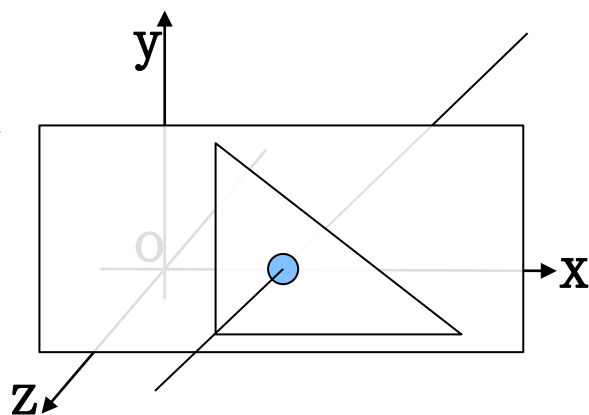
- 平面方程为

$$z = 3, \text{ 即 } (a, b, c, d) = (0, 0, 1, -3)$$

此时 $|c|$ 最大，取 oxy 为投影平面，利用 x, y 坐标进行计算

- 2D平面上如何判定点在三角形内？

- 直线方程
- 重心坐标
- 射线求交
- ...



细节3：反射方向计算

- 设 L 为射线方向, N 为射线与物体交点 P 处的法向

$$R_r = L - 2(L \cdot N)N$$

P : 入射光线 L 和物体的交点;

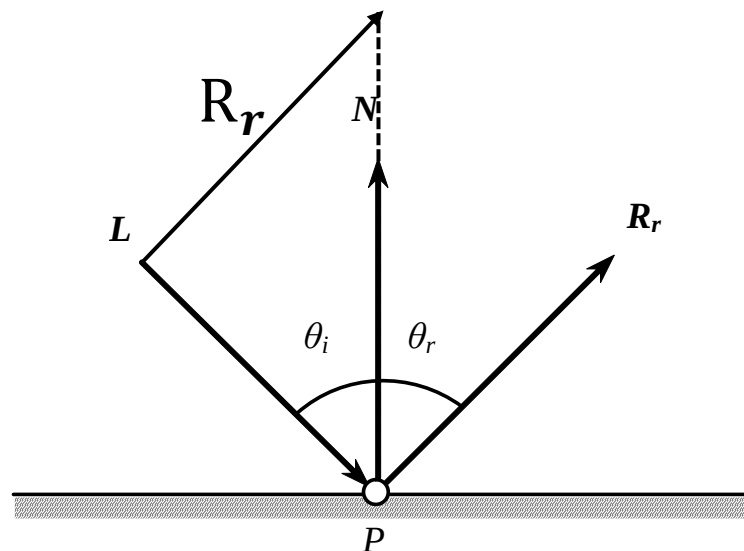
N : 点 P 处表面法向;

R_r : 镜面反射方向;

$\theta_i (= \theta_r)$: $-L$ 与 N 的夹角;

$\theta_r (= \theta_i)$: R_r 与 N 的夹角;

(注: L , N 均为单位矢量)



细节4：折射方向计算

- P : 入射光线 L 和物体的交点; N : P 处的物体表面法向

- R_t 为透射光线的方向

- Snell定律

(1) 介质1: 折射率 η_1 ; 介质2: 折射率 η_2

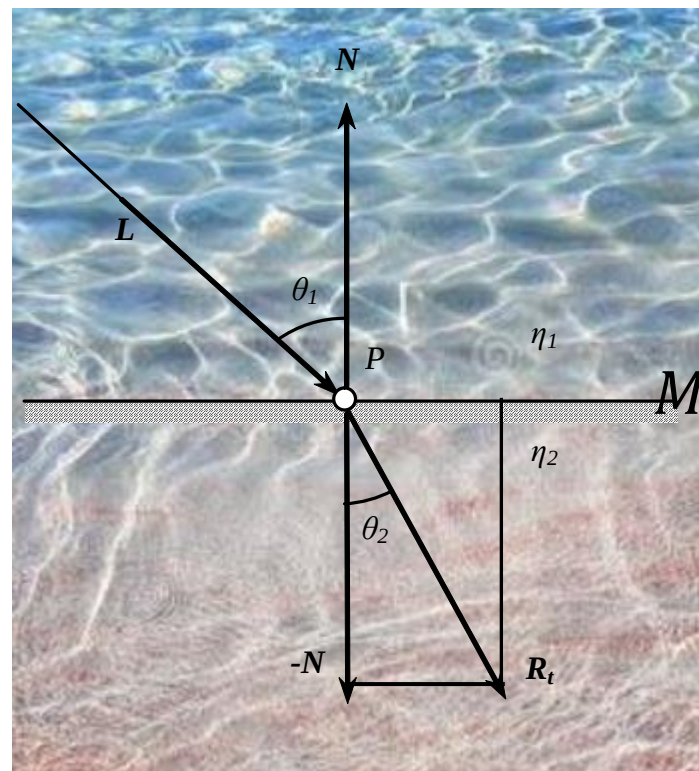
(2) L 与表面法向 N 的夹角为 θ_1

(3) R_t 与 N 的夹角为 θ_2 , 则有

$$\eta_1 \sin \theta_1 = \eta_2 \sin \theta_2$$

$$\rightarrow \sin \theta_2 = \frac{\eta_1 \sin \theta_1}{\eta_2}, \cos \theta_2$$

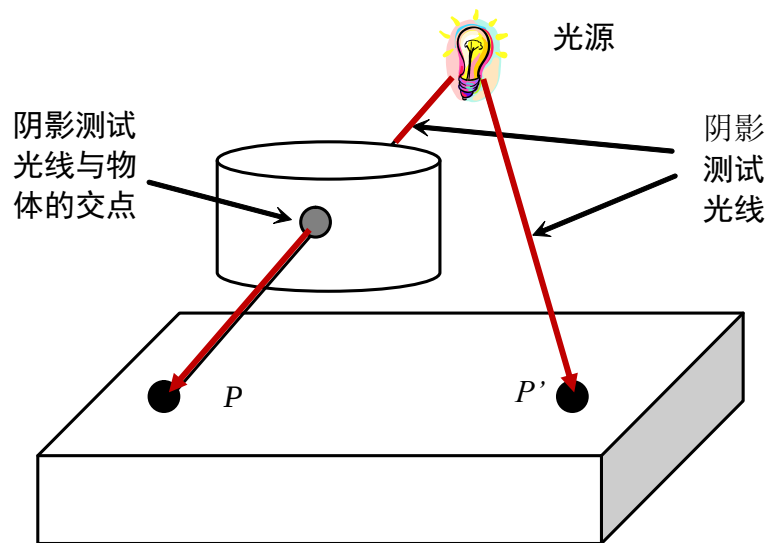
- $R_t = -N \cos \theta_2 + M \sin \theta_2$



细节5：局部光照 I_c 的阴影计算(shadow)

- 从P向光源L发射一条阴影测试光线R
 - 若R与途中的物体不相交，则点P受光源L直接照射
 - 否则，点P位于光源阴影之中

注：面光源可以用离散的点光源近似。



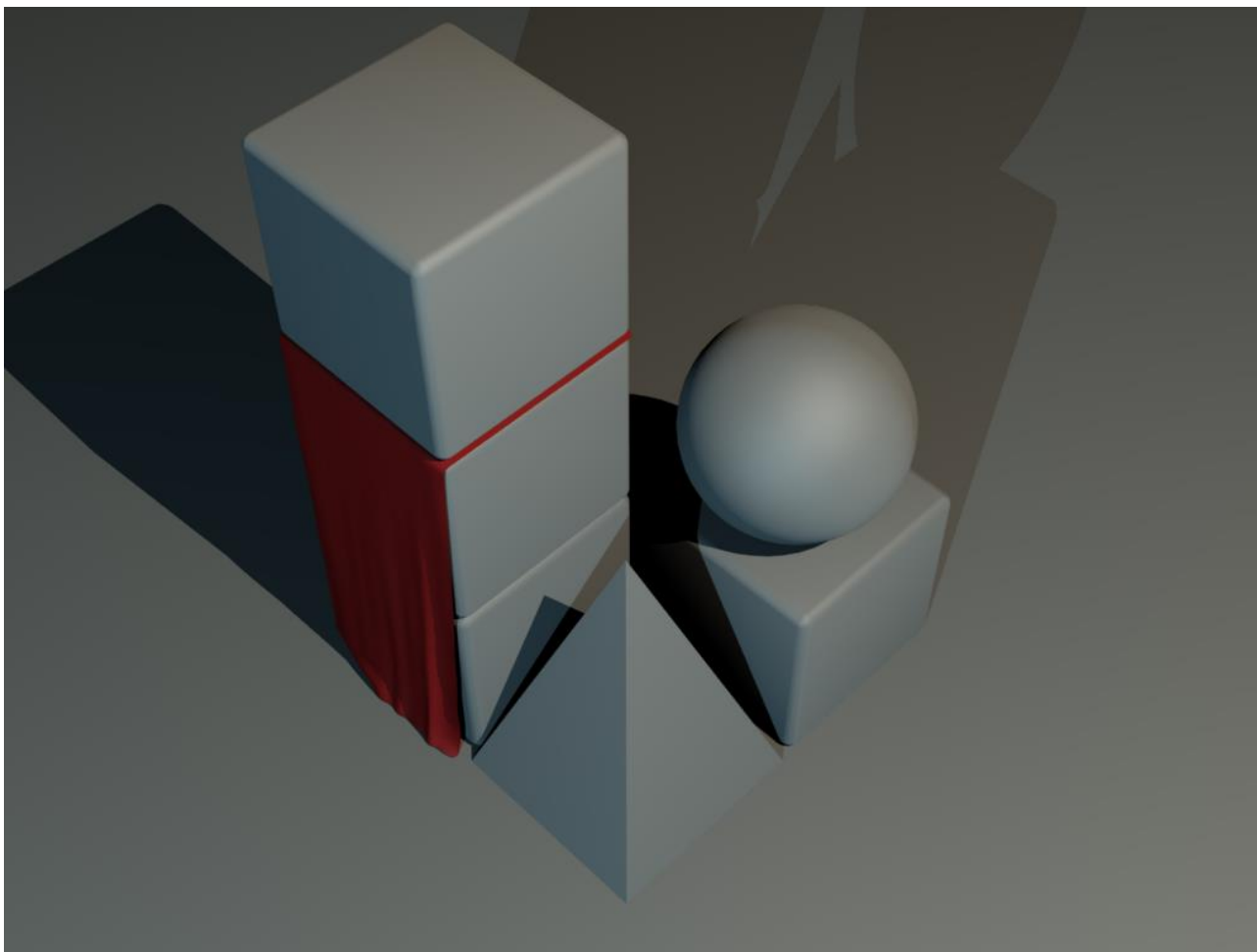
细节6：阴影计算

- 包含阴影计算的Phong模型

$$I(P) = K_a I_a + \sum_{i=1}^m \{f_i(P) I_i [K_d (N \cdot L_i) + K_s (N \cdot H_i)^n]\}$$

$$f_i(P) = \begin{cases} 1 & \text{点P受光源}i\text{直接照射} \\ 0 & \text{点P未受光源}i\text{直接照射} \end{cases}$$

光线跟踪阴影效果



小结：光线跟踪中的四类光线

- 视线Eye ray

- 从视点发出

- 光源线Shadow ray

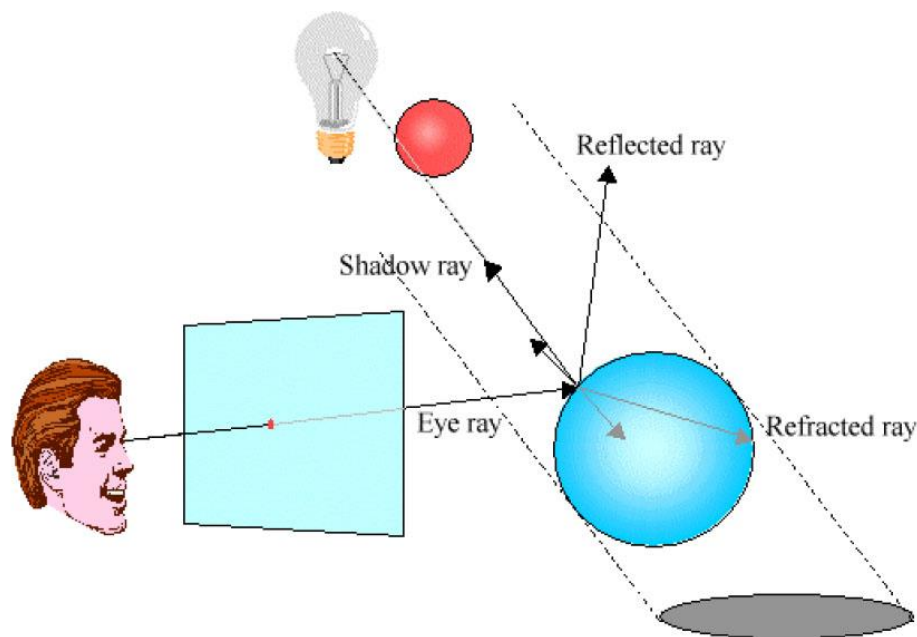
- 从物体表面上的点向光源发出

- 反射线Reflected ray

- 从物体表面上的点沿镜面反射方向发出

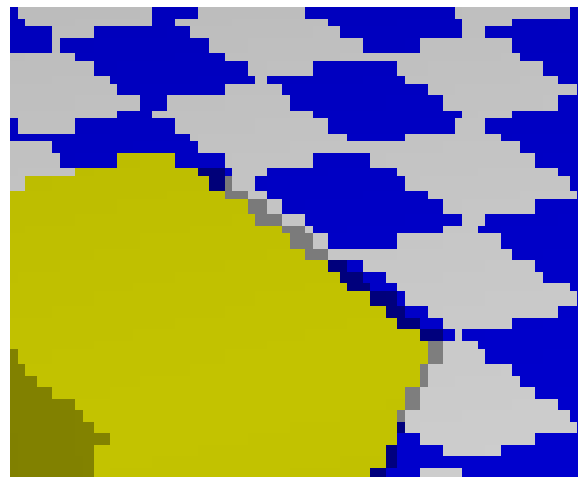
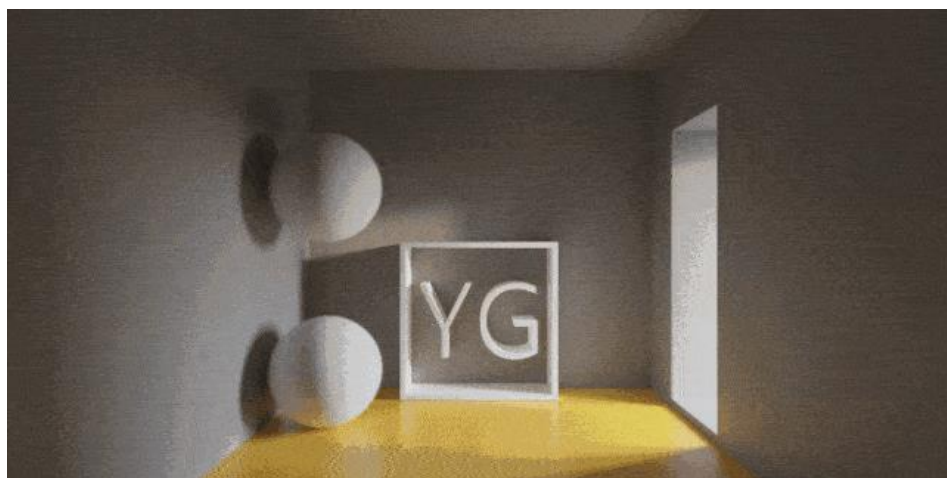
- 折射线Refracted ray

- 从物体表面上的点沿透射方向发出



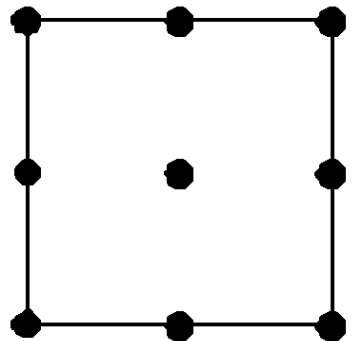
走样(Antialias)

- 引起走样的原因
 - 光线跟踪算法本质上是对画面的点采样
- 示例

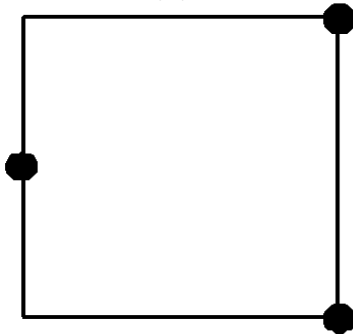


反走样(Antialias)处理方法

- 超采样：一个像素发出多条射线(rays)
- 自适应超采样：几何细节丰富的地方多发射线



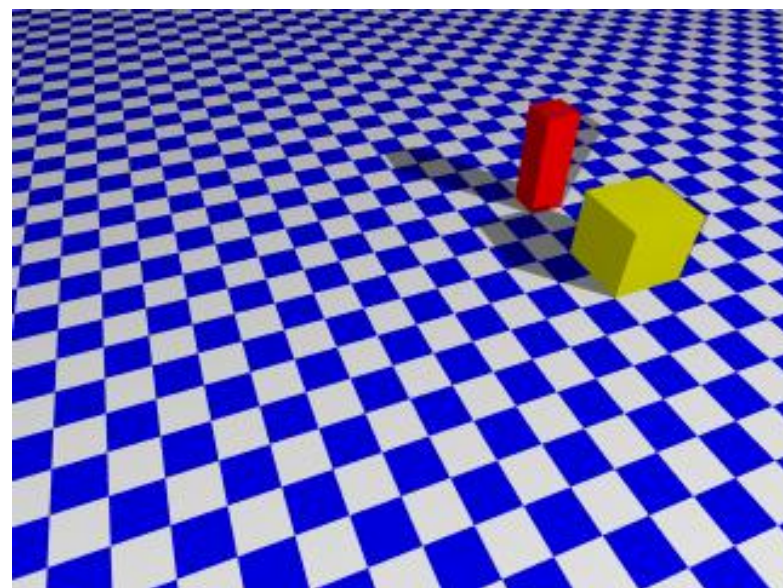
(a)



(b)

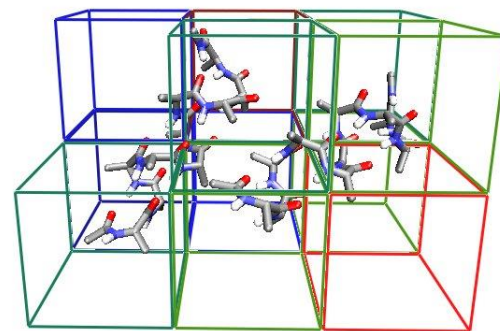
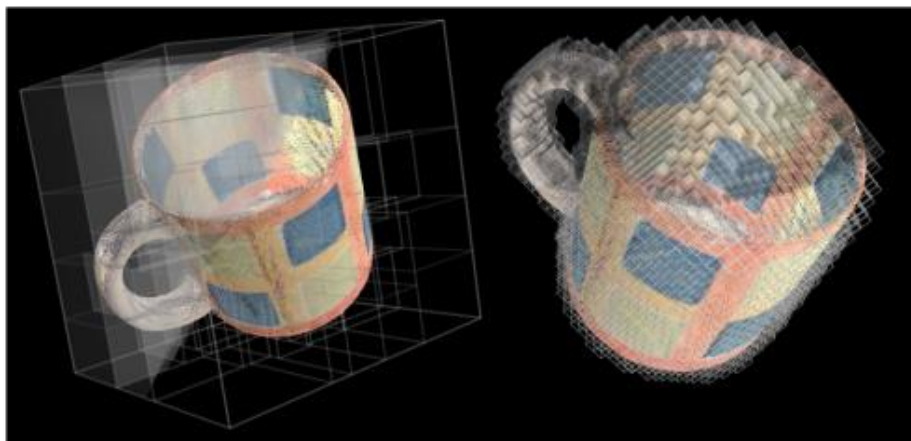
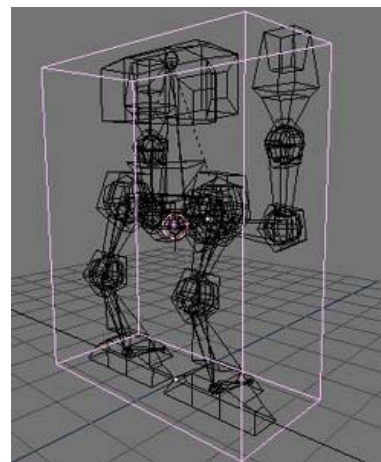


反走样效果示例



加速(Acceleration)

- 为什么要进行加速？
- 常用加速技术
 - 包围盒技术
 - 层次包围盒 (Bounding Volume Hierarchies BVH)
 - 空间分割技术

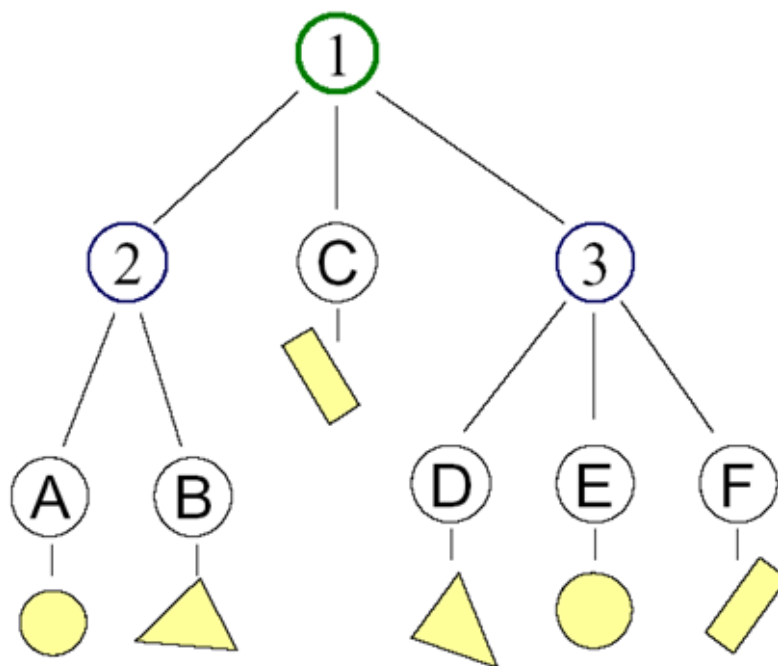
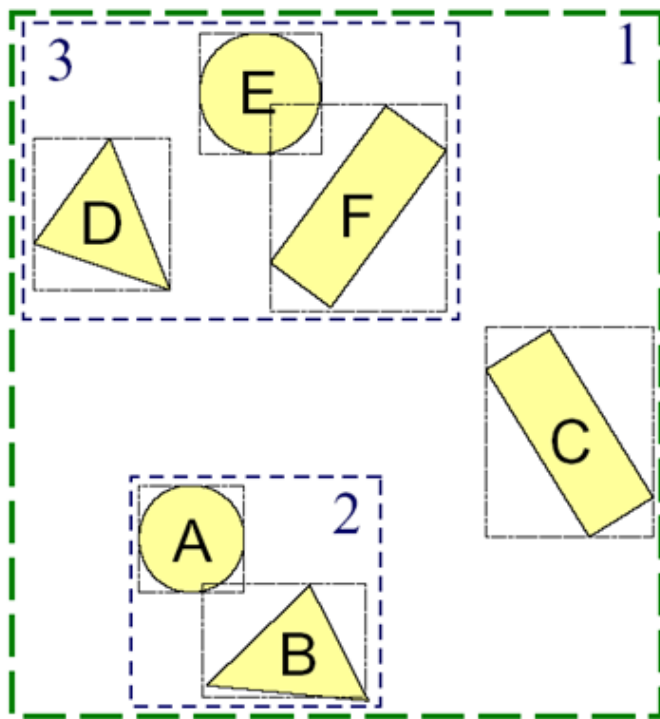


包围盒(Bounding box)

- 将场景按空间位置关系分层次组织成树状结构
 - 根结点：整个场景
 - 中间结点：空间位置较为接近的一组表面
 - 叶结点：单个景物表面
- 树结点中的面片集都用简单包围盒包裹
- 光线与包围盒有交时，才进行光线与其中所含的景物面片求交运算
 - 光线与包围盒不相交，必定不与其中所含的景物面片相交

层次包围盒(Hierarchical bounding box)

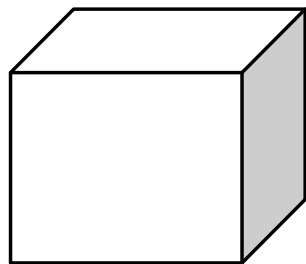
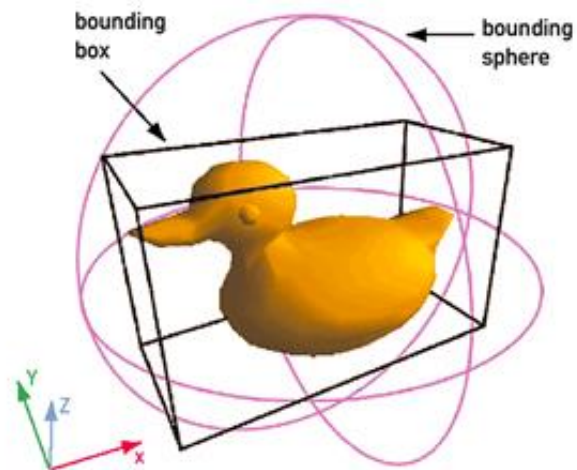
- 层次包围盒示例



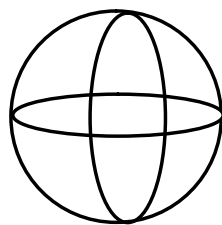
其他包围盒

● 常用包围盒

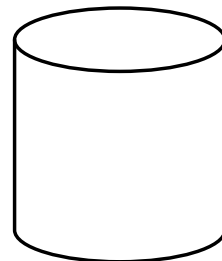
- 长方形包围盒
- 包围球
- 包围圆柱
- 平行 $2n$ 面体



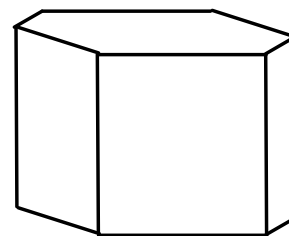
(a)



(b)



(c)

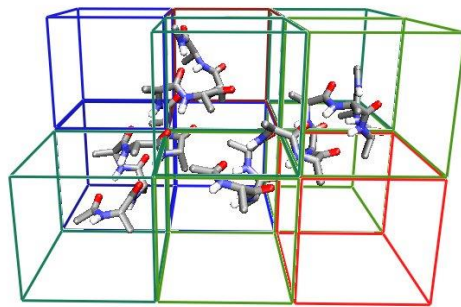


(d)

空间分割技术

● 原理

- 将景物空间分割成一个个小的空间单元
- 被跟踪的光线仅与它所穿过空间单元中所含物体表面进行求交测试
- 利用相邻空间单元的空间连贯性，使光线快速跨越空单元，迅速到达非空单元，求得光线与景物的第一个交点



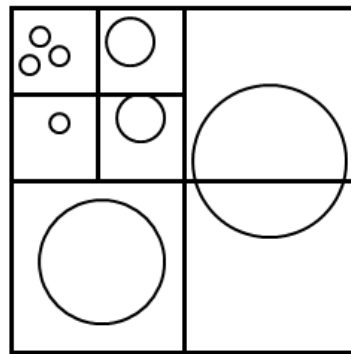
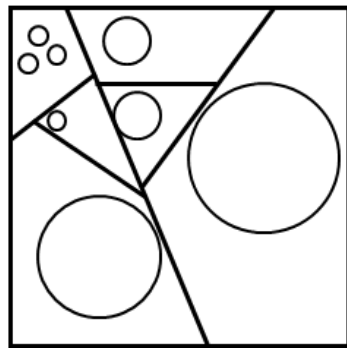
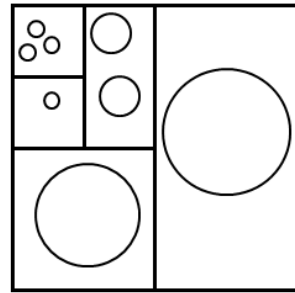
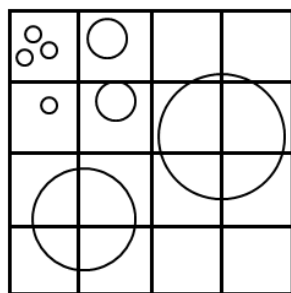
空间分割技术

● 典型方法

- 均匀网格：四叉树（二维）/八叉树（三维）

- Kd树

- BSP树



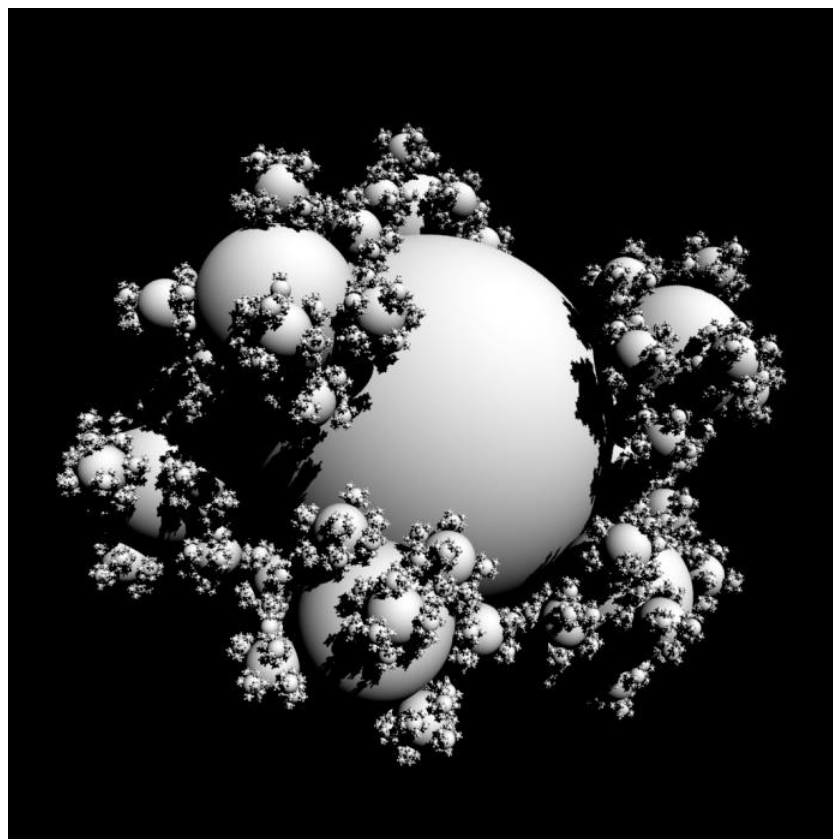
一个光线跟踪器：POV-Ray

- 开源光线跟踪软件POV-Ray简介
- <http://www.povray.org>



一个光线跟踪器：POV-Ray

- Berger-Perrin T. The sphere flake, in 100 lines of c code.
- <http://ompf.org/ray/sphereflake/>



J. Turner Whitted (Ph.D in 1978, NCSU)

- **Companies**

Microsoft Corporation, Numerical Design Limited, Bell Labs

- **Microsoft Corporation**

Senior Researcher, Hardware Devices and Graphics

- **Movies and computer games**

In the lineage of **Woody, Buzz, Shrek and the Matrix's Neo**, ..., a lot of characters and effects.

- A private pilot and an avid sailor.

- The CG Achievement Award

1986. ACM Fellow; National Academy of Engineering.

